

FactLens: Evidence-Grounded Verification of Claims in YouTube Videos

A project completed in partial fulfillment of the requirements for
the award of the degree of

MASTER OF SCIENCE

in

COMPUTER SCIENCE

By

YUVRAJ SINGH

(205324026)



**DEPARTMENT OF COMPUTER APPLICATIONS
NATIONAL INSTITUTE OF TECHNOLOGY
TIRUCHIRAPALLI-620015**

JUNE 2026

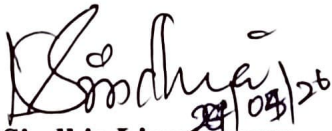
BONAFIDE CERTIFICATE

This is to certify that the project titled **FactLens: Evidence-Grounded Verification of Claims in YouTube Videos** is a bonafide record of the work done by

YUVRAJ SINGH

(205324026)

in partial fulfillment of the requirements for the award of the degree of **Master of Science in Computer Science** of the **National Institute of Technology, Tiruchirappalli**, during the year 2025-2026 (4th Semester-CAS750 Project Work Phase-II).



Dr. Sindhia Lingaswamy
Guide



Dr. S. Domnic
Head of the Department

Head of the Department
Dept. of Computer Applications
National Institute of Technology
Tiruchirappalli - 620 015.
Tamilnadu, India.

Project Viva-voice held on _____

Internal Examiner

External Examiner

ABSTRACT

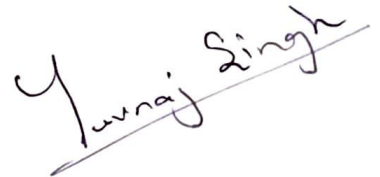
The growing consumption of video content on platforms like YouTube has created a significant challenge in combating misinformation, as viewers rarely have the time or resources to verify claims made in videos while watching them. FactLens addresses this gap by providing an automated, real-time claim verification system built as a Chrome extension. Once enabled, the extension continuously captures audio from the playing video in timed chunks, each tagged with its corresponding timestamp, and transmits them to a backend pipeline for processing. The pipeline first transcribes the audio into text, then employs a large language model to extract only meaningful, verifiable claims while deliberately excluding opinions, jokes, personal anecdotes, and other content that does not warrant factual scrutiny. Each identified claim is then subjected to an evidence gathering phase that draws from a diverse set of sources including Wikipedia, news APIs, Google Search, ArXiv, and AI-powered retrieval APIs, ensuring that the verification is grounded in broad and reliable information. A language model then weighs the gathered evidence against each claim and assigns a verdict of Support, Contradict, or Inconclusive, accompanied by a concise explanation of its reasoning. These results are surfaced to the viewer through a non-intrusive overlay on the video interface, presenting the timestamp, the claim, the verdict, and the rationale in a clear and readable format. By embedding fact-checking directly into the viewing experience, FactLens makes verification effortless and accessible, empowering viewers to engage more critically with the content they consume.

ACKNOWLEDGEMENT

I express my sincere thanks to **Dr. G. Aghila**, Director, National Institute of Technology, Tiruchirappalli, for permitting me to undertake this project.

I wish to record my sincere thanks to **Dr. S.Domnic**, Head, Department of Computer Applications, National Institute of Technology, Tiruchirappalli. I would like to extend my deep gratitude to my internal guide, **Dr. Sindhia Lingaswamy**, Department of Computer Applications, National Institute of Technology, Tiruchirappalli for her whole hearted assistance not only for the duration of project but for the entire duration of my course. I would like to thank her for duly evaluating my progress and encouraging me.

Finally, I would like to express my regards for all the faculty members of Department of Computer Applications, friends and others involved in this project, directly or indirectly.



Yuvraj Singh

(205324026)

TABLE OF CONTENTS

CONTENT	PAGE NO.
ABSTRACT	3
ACKNOWLEDGEMENT	4
TABLE OF CONTENTS	5
LIST OF FIGURES	7
LIST OF ABBREVIATIONS	8
CHAPTER 1 INTRODUCTION	9
1.1 PROBLEM BACKGROUND AND MOTIVATION	9
1.2 OBJECTIVE AND SCOPE OF THE PROJECT	9
1.3 OVERVIEW OF THE PROPOSED SOLUTION	10
1.4 TOOLS AND TECHNOLOGIES USED	10
CHAPTER 2 LITERATURE REVIEW	12
2.1 RECENT STUDIES	12
2.2 REVIEW OF EXISTING SOTA MODELS	14
CHAPTER 3 SYSTEM ANALYSIS AND DESIGN	16
3.1 LIMITATIONS OF EXISTING SYSTEMS	16
3.2 USER REQUIREMENTS AND FUNCTIONAL GOALS	16
3.3 SYSTEM ARCHITECTURE OVERVIEW	17
CHAPTER 4 IMPLEMENTATION	19
4.1 PROTOTYPE OF CHROME EXTENSION FOR RECORDING YOUTUBE VIDEOS	19
4.2 BACKEND PIPELINE: AUDIO TRANSCRIPTION AND CLAIM DETECTION	21
4.3 EVIDENCE GATHERING, REASONING AND VERDICT GENERATION	24
4.4 EVALUATION ON THE FEVER DATASET	26
4.5 REAL-TIME OVERLAY DISPLAY AND USER INTERFACE	28
4.6 DEPLOYMENT: BACKEND TUNNELING AND CHROME EXTENSION DISTRIBUTION	30

CHAPTER 5 RESULTS AND DISCUSSION	32
5.1 PIPELINE PERFORMANCE	32
5.2 END-TO-END SYSTEM BEHAVIOUR	32
5.3 OBSERVED STRENGTHS	33
5.4 LIMITATIONS AND CHALLENGES	34
5.5 DISCUSSION	34
CHAPTER 6 CONCLUSION	36
REFERENCES	37

LIST OF FIGURES

Fig. No.	Title	Page No.
1	FactLens Extension in Action	21
2	Audio Transcription and Claim Detection Pipeline	23
3	Evidence Gathering and Verdict Generation Pipeline	26
4	FactLens Verification Overlay on YouTube	29

LIST OF ABBREVIATIONS

LLM	Large Language Model
SOTA	State of the Art
SERP	Search Engine Results Page
MV3	Manifest Version 3
WAV	Waveform Audio File Format
UI	User Interface
UX	User Experience
API	Application Programming Interface
CSS	Cascading Style Sheets
HTML	Hypertext Markup Language
HTTP	Hypertext Transfer Protocol
HTTPS	Hypertext Transfer Protocol Secure
JS	JavaScript
JSON	JavaScript Object Notation
PCM	Pulse Code Modulation
URL	Uniform Resource Locator

CHAPTER 1

INTRODUCTION

1.1 PROBLEM BACKGROUND AND MOTIVATION:

The internet has fundamentally changed how people consume information, with video content becoming one of the most influential mediums in shaping public opinion. YouTube alone hosts billions of videos spanning news, education, politics, health, and science, making it a primary source of information for millions of users worldwide. However, this scale also makes it a breeding ground for misinformation, where false or misleading claims can reach vast audiences before any correction is made. Unlike text-based content, video is harder to fact-check because the information is spoken rather than written, making it difficult to quickly scan or verify. Viewers typically watch passively and rarely pause to investigate the accuracy of what is being said. Existing fact-checking tools are largely manual, reactive, and focused on written content, leaving a clear gap when it comes to real-time verification of spoken claims in videos. This creates a need for an automated system that can work alongside the viewer, identifying and verifying claims as the video plays without requiring any additional effort from the user. FactLens is motivated by this need, aiming to bring passive, evidence-grounded fact-checking directly into the video watching experience.

1.2 OBJECTIVE AND SCOPE OF THE PROJECT:

The primary objective of FactLens is to develop a real-time, automated claim verification system that works directly within the YouTube viewing experience through a Chrome extension. The system aims to transcribe spoken audio from videos, extract meaningful and verifiable claims using a large language model, gather supporting evidence from multiple reliable sources, and deliver a clear verdict to the viewer without interrupting their experience. The project also focuses on ensuring that only factually checkable claims are processed by filtering out opinions, jokes, and personal statements, improving the overall accuracy and relevance of the results presented.

The scope of the project is limited to YouTube videos accessed through the Google Chrome browser. The system processes audio in real time by capturing timed chunks and routing them through a backend pipeline, covering transcription, claim detection, evidence retrieval, and verdict generation. Evidence is gathered from sources including Wikipedia, news APIs, Google Search, ArXiv, and AI-powered APIs. The final output is displayed as an overlay on

the video page, showing the timestamp, claim, verdict, and reasoning. The project does not cover video formats outside of YouTube, nor does it address non-English content in its current scope.

1.3 OVERVIEW OF THE PROPOSED SOLUTION:

FactLens is built as a Chrome extension using the Manifest V3 [1] template provided by the Chrome browser framework. The extension's frontend is developed using HTML, CSS, and JavaScript, covering the popup interface, the video overlay, and the overall user interaction layer. Coordination between the different components of the extension is managed by a background service worker, which handles communication between the content script and the backend. The content script injected into the YouTube page is responsible for capturing audio chunks from the playing video at regular intervals along with their timestamps and forwarding them to the backend for processing.

On the backend, audio transcription is performed using OpenAI's Whisper base model [2], which converts the spoken content into text. The transcribed text is then passed to the Llama 3.3 70B Versatile model [3], which identifies verifiable claims from the content while filtering out opinions, jokes, personal anecdotes, and other statements that are not suitable for fact-checking. Once claims are extracted, evidence is gathered from a range of sources including Wikipedia, ArXiv, SerpAPI [4], Tavily [5], and NewsAPI, ensuring coverage across general knowledge, academic literature, web search results, and current news. The same Llama 3.3 70B Versatile model is then used to reason over the gathered evidence and assign a verdict of Support, Contradict, or Inconclusive to each claim, along with a brief explanation. The verified results are sent back to the extension and displayed as a non-intrusive overlay on the YouTube page, showing the timestamp, the claim, the verdict, and the reasoning behind it.

1.4 TOOLS AND TECHNOLOGIES USED:

- Chrome Extension Framework: Manifest V3 template with HTML, CSS, and JavaScript for the UI and extension components.
- Background Service Worker: Manages coordination between the content script and the backend.
- Speech to Text: OpenAI Whisper base model for audio transcription.

- Claim Detection and Verdict Generation: Llama 3.3 70B Versatile model for both extracting verifiable claims and reasoning over evidence.
- Evidence Gathering APIs: Wikipedia, ArXiv, SerpAPI, Tavily, and NewsAPI.
- Backend: Python based pipeline handling transcription, claim detection, evidence retrieval, and verdict generation.
- Deployment: The backend is served using Uvicorn [6] and exposed publicly using Ngrok [7], which provides a secure HTTPS tunnel for the Chrome extension to communicate with. The Chrome extension is loaded into the browser using Chrome's Developer Mode via Load Unpacked.

CHAPTER 2

LITERATURE REVIEW

2.1 RECENT STUDIES:

The problem of misinformation in online video content has attracted growing research attention in recent years, with studies approaching it from different angles including multimodal learning, live stream analysis, hybrid detection, and text-based fact-checking. While each of these works has contributed meaningfully to the field, they collectively highlight a gap that FactLens aims to address.

Rui Cao (2025) proposed an approach for detecting misinformation in live streaming videos by directly processing raw audio-visual signals in real time [8]. The strength of this work lies in its ability to operate on live content as it is being streamed, which is a technically challenging problem. The system processes the combined audio and visual feed to detect potential misinformation without requiring a transcript or any preprocessing. However, the approach does not attempt to identify or isolate specific claims being made within the video. It treats the content as a whole rather than breaking it down into individual statements that can be independently verified. Furthermore, the system does not cross-reference the content against real-world facts from external sources, meaning its judgments are not grounded in evidence. No explanation is provided to the viewer about what was flagged or why, which limits the system's usefulness in helping users understand and evaluate the content they are watching.

Yang et al. (2025) presented a more sophisticated approach to fake news detection in short videos by combining deep multimodal learning with semantic credibility analysis and multi-granularity contrastive learning [9]. The model is designed to assess the credibility of video content by learning from multiple modalities including text, audio, and visual features, and it produces interpretable credibility scores that offer some degree of transparency. This is a notable improvement over black-box detection systems. However, the model assigns a single fake or real label to the entire video rather than identifying and verifying specific claims within it. This limits its practical usefulness because a video may contain a mix of accurate and inaccurate statements, and a single overall label fails to capture that nuance. Additionally,

the evidence or reasoning behind the credibility score is not surfaced to the end user, leaving viewers without the context they need to critically evaluate the content themselves.

Kim et al. (2025) introduced a hybrid detection method specifically designed for YouTube fake news, which leverages related video data alongside the target video to provide broader contextual understanding [10]. By incorporating information from multiple related videos, the model outperforms single-video detection approaches and demonstrates that contextual signals can significantly improve detection accuracy. Despite this, the system has several practical limitations. It relies on other videos as its primary source of contextual evidence rather than drawing from authoritative or trusted external sources such as news outlets, encyclopedias, or academic databases. The system also lacks real-time processing capability, meaning it cannot be used while a video is being watched. Most significantly, it operates at the video level and does not perform claim-level reasoning, making it impossible to pinpoint which specific statements within a video are accurate or misleading.

Text-based fact-checking systems including FEVER, LIAR, and ClaimBuster have laid important groundwork for automated claim verification by demonstrating that language models and structured datasets can be used to effectively verify written claims against evidence [11,12,13]. These systems employ evidence retrieval and natural language inference to determine whether a given claim is supported or refuted by a body of text. Their claim-level approach and evidence-grounded decision making are directly relevant to the goals of FactLens. However, their fundamental limitation is that they are designed exclusively for written text. They have no mechanism for processing audio or video content, cannot handle the temporal nature of video where claims are made at specific points in time, and are not designed to operate in a real-time or browser-based setting. This makes them unsuitable for direct application to the YouTube video fact-checking problem despite their strong performance on text benchmarks.

Overall, these studies reveal a consistent pattern of limitations. Systems that work on video often lack claim-level granularity, evidence grounding, or real-time capability. Systems that perform strong claim verification are confined to text and cannot be applied to spoken content in videos. FactLens is designed to bridge this gap by combining real-time audio processing, specific claim extraction, multi-source evidence retrieval, and transparent verdict generation into a single pipeline that works directly within the viewing experience on YouTube.

2.2 REVIEW OF EXISTING SOTA MODELS:

Beyond individual research studies, several state-of-the-art models have been developed to tackle misinformation and fake news detection in video content. While these models represent significant technical progress, a closer look at their design reveals limitations that make them insufficient for the kind of claim-level, evidence-grounded verification that FactLens aims to deliver.

FakeSV-VLM is a multimodal model that processes both video and text to detect fake news in short video content [14]. Its core strength lies in its ability to capture inconsistencies between what is shown visually and what is said or described in text, making it one of the more capable models for understanding video content holistically. The model benefits from the representational power of vision-language architectures, which allow it to reason across different modalities simultaneously. However, FakeSV-VLM produces only a video-level label, classifying the entire video as fake or real without identifying which specific claims within the video are problematic. It does not perform any form of fact-checking against external sources, meaning its judgments are based purely on learned patterns rather than verified evidence. The model also does not operate in real time, limiting its usefulness in a live or active viewing context.

VMID, which employs a large language model fusion approach, demonstrates that LLM-based reasoning can be effectively applied to video misinformation detection, achieving good accuracy on benchmark datasets [15]. The use of language model reasoning gives it an advantage in understanding the semantic content of videos and drawing meaningful inferences. Despite this, VMID is designed for offline use and cannot process videos as they are being watched. It does not retrieve or consult trusted external sources when making its decisions, which means its verdicts are not backed by verifiable evidence. Like FakeSV-VLM, it also provides no explanation to the viewer, making it difficult for users to understand or question the model's conclusions.

The Consistency-Aware Model takes a different approach by focusing on detecting contradictions across multiple modalities within a video, identifying cases where the visual content, speech, and text signals are inconsistent with one another [16]. This cross-modal contradiction detection is a useful capability for spotting certain types of manipulated or misleading content. However, the model does not extract individual claims from the video

content, meaning it cannot point to a specific statement that is false or misleading. It also performs no external verification, relying entirely on internal consistency signals rather than comparing content against real-world facts from authoritative sources.

Across all three models, a common set of shortcomings emerges. None of them perform claim-level extraction or verification, none consult trusted external sources to ground their decisions in real-world evidence, and none are designed for real-time use during active video playback. FactLens directly addresses each of these gaps by extracting specific verifiable claims from spoken audio, gathering evidence from multiple trusted sources, and delivering time-stamped, evidence-backed verdicts to the viewer in real time through a lightweight browser extension.

CHAPTER 3

SYSTEM ANALYSIS AND DESIGN

3.1 LIMITATION OF EXISTING SYSTEMS:

A review of existing misinformation detection approaches reveals several recurring gaps that make them insufficient for practical, real-time claim verification in YouTube videos.

- Most existing systems are designed for offline use and cannot process video content in real time while it is being watched.
- Existing models classify content at the video level, producing a single fake or real label rather than identifying and verifying individual claims.
- Decisions are based on patterns learned during training rather than evidence retrieved from trusted external sources, making verdicts unreliable for recent or domain-specific claims.
- Little to no explanation is provided to the viewer, making it difficult to understand or trust the system's output.
- Text-based fact-checking systems like FEVER, LIAR, and ClaimBuster cannot handle audio or video input and are not suitable for verifying spoken claims in YouTube videos.
- None of the reviewed systems are integrated into the browser or the video watching experience, requiring users to manually submit content to separate tools which is impractical for everyday use.
- Systems that work on video such as FakeSV-VLM, VMID, and the Consistency-Aware Model do not consult external sources, do not extract specific claims, and do not provide reasoning behind their verdicts.

These limitations collectively highlight the need for a system that is real-time, claim-specific, evidence-grounded, and transparent, all of which FactLens is built to deliver.

3.2 USER REQUIREMENTS AND FUNCTIONAL GOALS:

Based on the limitations identified in existing systems and the core purpose of FactLens, the following user requirements and functional goals were established to guide the design and development of the system.

User Requirements:

- The system should work directly within the YouTube viewing experience without requiring the user to switch to a separate tool or platform.
- The extension should be easy to enable and disable, requiring minimal setup or technical knowledge from the user.
- Claim verification results should be displayed in a clear and readable format that does not disrupt the viewing experience.
- Each result should show the timestamp at which the claim was made, the claim itself, the verdict, and the reasoning behind it.
- The system should process video content automatically without requiring any manual input from the user during playback.
- The results should be trustworthy, meaning each verdict must be backed by evidence gathered from reliable external sources.

Functional Goals:

- Capture audio from YouTube videos in timed chunks and send them to the backend along with their corresponding timestamps.
- Transcribe the captured audio accurately into text using a reliable speech to text model.
- Extract only meaningful and verifiable claims from the transcribed text, filtering out opinions, jokes, personal statements, and irrelevant content.
- Retrieve supporting evidence for each extracted claim from multiple trusted sources including Wikipedia, ArXiv, SerpAPI, Tavily, and NewsAPI.
- Analyse the gathered evidence against each claim and generate a verdict of Support, Contradict, or Inconclusive along with a concise explanation.
- Return the verified results to the Chrome extension and display them as a non-intrusive overlay on the YouTube page.
- Ensure the backend pipeline is accessible and reliable by serving it through Uvicorn and exposing it via ngrok for secure communication with the Chrome extension.

3.3 SYSTEM ARCHITECTURE OVERVIEW:

FactLens follows a client-server architecture where the Chrome extension acts as the frontend client and a Python-based backend server handles all the heavy processing. The two

components communicate over HTTP, with the extension sending audio data to the backend and receiving verified claim results in return.

Chrome Extension (Frontend):

The frontend consists of three main components working together. The popup interface allows the user to enable or disable the extension with a simple toggle. The content script is injected into the YouTube page and is responsible for capturing audio from the playing video in regular timed chunks, each paired with its corresponding timestamp. The background service worker acts as the coordinator, managing communication between the content script and the backend server, ensuring chunks are sent and results are received in an organised manner. Once results arrive, the content script renders them as an overlay directly on the YouTube page.

Backend Pipeline:

The backend is a Python server that receives audio chunks from the extension and processes them through a sequential pipeline. The first stage is transcription, where the Whisper base model converts the audio into text. The transcribed text is then passed to the Llama 3.3 70B Versatile model, which identifies verifiable claims while filtering out content that does not warrant fact-checking. Each extracted claim then enters the evidence gathering stage, where relevant information is retrieved from Wikipedia, ArXiv, SerpAPI, Tavily, and NewsAPI in parallel. Finally, the same Llama model reasons over the gathered evidence and assigns a verdict of Support, Contradict, or Inconclusive to each claim along with a brief explanation. The results are then packaged and sent back to the extension.

Deployment:

The backend is served using Uvicorn and exposed through ngrok, which provides a secure HTTPS tunnel enabling the Chrome extension to communicate with the backend in real time. The Chrome extension is deployed directly into the browser using Chrome's Developer Mode, making the setup lightweight and straightforward without requiring any cloud infrastructure.

The overall flow can be summarised as follows: the extension captures audio, the backend transcribes, extracts, verifies, and responds, and the extension displays the results to the viewer in real time.

CHAPTER 4

IMPLEMENTATION

4.1 PROTOTYPE OF CHROME EXTENSION FOR RECORDING YOUTUBE VIDEOS:

The Chrome extension serves as the entry point of the FactLens pipeline, responsible for capturing audio from YouTube videos and initiating the verification process. It is built using the Manifest V3 framework and consists of three core components working together: the popup interface, the content script, and the background service worker.

Manifest Configuration:

The extension is configured through a manifest file which declares the necessary permissions including access to the active tab and storage, along with host permissions for YouTube and the backend URL. The content script is set to run after the page is fully loaded on all YouTube pages, ensuring it initializes only when the page is ready. The background service worker is registered separately to handle coordination tasks.

Popup Interface:

The popup is the user-facing part of the extension, accessible by clicking the extension icon in the Chrome toolbar. It presents two buttons, Start Capture and Stop Capture, which allow the user to control when audio recording begins and ends. Clicking Start Capture triggers a message to the background service worker, which then forwards the instruction to the content script running on the YouTube page.

Content Script:

The content script is injected directly into the YouTube page. When it receives the Start Capture instruction, it locates the video element on the page and waits until it is available before proceeding. It then sets up an audio context at a sample rate of 16000 Hz and connects the video element as an audio source. The audio processor continuously collects raw audio samples and once the buffer accumulates 10 seconds worth of audio, it converts the data and sends it to the background service worker along with the current video timestamp. When Stop Capture is received, the audio processor is disconnected and collection halts.

Background Service Worker:

The background service worker acts as the coordinator between the content script and the backend. When it receives an audio chunk, it converts and encodes it into a proper WAV file format and packages it along with the chunk start timestamp and a unique session ID. This is then sent as a POST request to the backend endpoint. Once the backend responds with the verified claims, the service worker forwards the results back to the content script to be displayed.

Overlay Display:

Upon receiving the results, the content script renders them as a fixed overlay on the YouTube page. The overlay appears in the top right corner of the screen and displays each verified claim in a card format showing the claim text, the timestamp at which it was made, the verdict in colour coded text where green indicates Support, red indicates Contradict, and white indicates Inconclusive, along with a brief explanation of the reasoning. New claims are prepended to the top of the overlay so the most recent result is always visible first. The overlay also includes a close button allowing the user to dismiss it at any time.

Flow Summary:

The overall flow is straightforward. The user plays a video on YouTube, enables the extension, and clicks Start Capture. From that point, audio chunks with their corresponding timestamps are automatically sent to the backend every 10 seconds. The backend processes each chunk and returns the verified claims, which are immediately displayed on the overlay without any further action required from the user. Clicking Stop Capture ends the audio collection at any point.

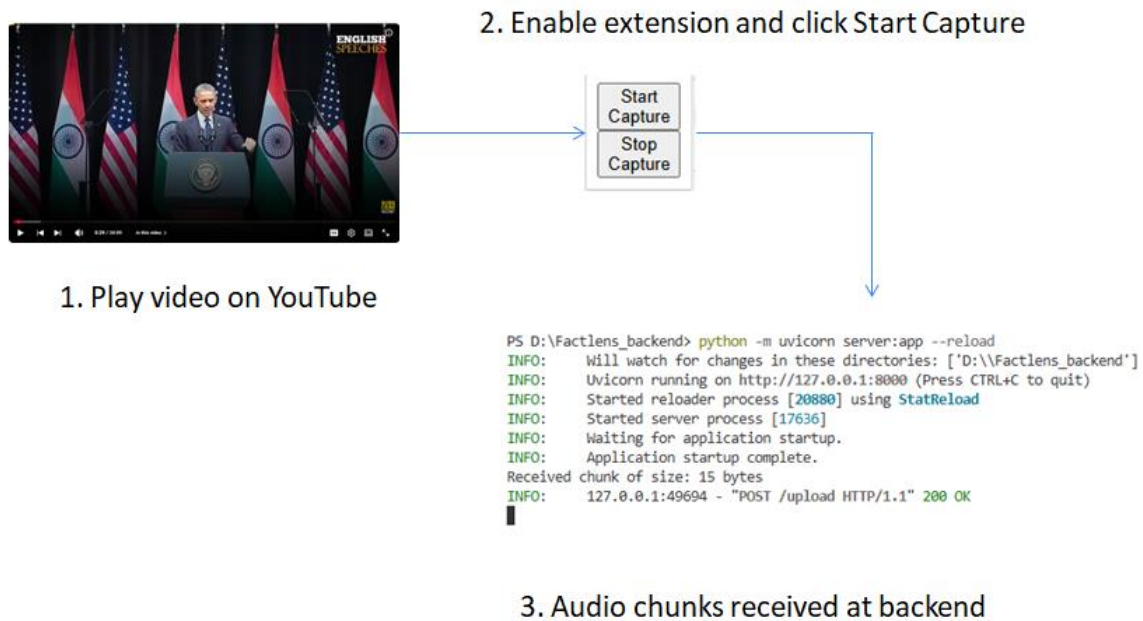


Fig 1: FactLens Extension in Action

The figure illustrates the end-to-end flow of the FactLens Chrome extension, showing a YouTube video being played, the Start and Stop Capture controls in the popup interface, and the backend receiving the audio chunks for processing.

4.2 BACKEND PIPELINE: AUDIO TRANSCRIPTION AND CLAIM DETECTION:

The backend is built using FastAPI and serves as the core processing engine of the FactLens pipeline. It exposes a single endpoint that accepts incoming audio chunks from the Chrome extension along with two additional pieces of information: the timestamp marking the start of that audio chunk in the video, and a unique session identifier that is generated once when the user begins capture and remains consistent across all subsequent chunks from the same viewing session.

Receiving the Audio:

When the backend receives a request, it first checks that the required API keys are available in the environment. If any key is missing, it returns an error immediately without proceeding further. If everything is in order, the incoming audio data is read from the request and written to a temporary WAV file on disk so it can be passed to the transcription model. A session record is also initialized on first contact, which stores two things for the duration of the

session: a rolling context window of recent transcript segments, and a list of claims that have already been extracted, both of which are used in later stages.

Audio Transcription using Whisper:

The temporary WAV file is passed to the Whisper base model, which is loaded once when the server starts and kept in memory to avoid reloading delays on every request. Whisper processes the audio and returns a list of segments, where each segment contains the spoken text and its relative start and end timestamps within the chunk. Since these timestamps are relative to the beginning of the audio chunk rather than the full video, they are converted into absolute video timestamps by adding the chunk start time that was sent along with the audio. This ensures that every transcribed segment can be precisely mapped back to the exact moment in the video where it was spoken, which is important for displaying accurate timestamps in the overlay later. Once transcription is complete, the temporary audio file is deleted from disk.

Claim Detection using Llama 3.3 70B Versatile:

Each transcribed segment is processed individually by the Llama 3.3 70B Versatile model, accessed through the Groq API [17], which enables fast inference. The model is given a carefully designed prompt that instructs it to act as a factual claim extraction system. It is told to extract only statements that are checkable, meaning they can be verified as true or false against real-world information. Statements about public figures, countries, organisations, and events are considered valid for extraction even if they might turn out to be false, because the goal is to identify claims worth verifying rather than pre-judging their accuracy.

The model is explicitly instructed to avoid extracting pure opinions or personal beliefs, jokes, sarcasm, figurative language, exaggerations, metaphors, and vague or generic statements that cannot be meaningfully fact-checked. This filtering is critical because a significant portion of spoken content in YouTube videos falls into these categories, and including them would produce irrelevant or misleading verification results.

Context Maintenance and Co-reference Resolution:

One of the key challenges in processing speech segment by segment is that speakers frequently use pronouns and references that only make sense in the context of what was said earlier. For example, a speaker might say "he discovered this in 1953" without the model knowing who "he" refers to unless the previous segments are also available. To address this,

a context window containing the last five transcript segments from the current session is maintained and included in every prompt sent to the model. This allows the model to resolve references correctly and extract claims with the proper subject, making the output more accurate and readable.

Duplicate Claim Filtering:

Since the video is processed in continuous chunks, the same claim or a very similar one may appear across multiple chunks, either because the speaker repeats themselves or because a chunk boundary splits a sentence that overlaps with the previous one. To handle this, every newly detected claim is compared against all previously extracted claims from the same session using a word similarity algorithm based on sequence matching. If the similarity between the new claim and any existing claim exceeds a threshold of 0.85, the new claim is treated as a duplicate and discarded. Only claims that are genuinely new and distinct are allowed to proceed to the evidence gathering stage, keeping the results clean and avoiding redundancy in the overlay.

Outcome of This Stage:

By the end of this stage, the backend has a list of unique, verifiable claims extracted from the audio chunk, each tagged with its absolute start and end timestamps in the video. These claims are then passed forward to the evidence gathering and verification stage, which cross-references each one against multiple trusted external sources before producing a final verdict.

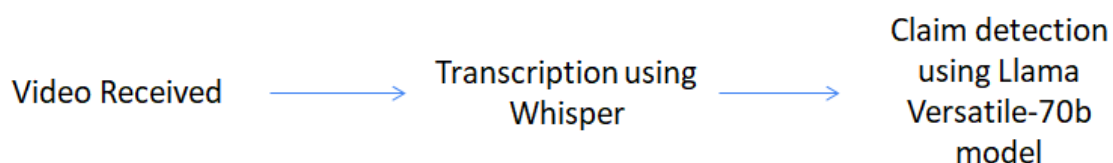


Fig 2: Audio Transcription and Claim Detection Pipeline

The figure illustrates the first half of the backend pipeline, showing how a received audio chunk is transcribed using Whisper and passed to the Llama 3.3 70B Versatile model for claim detection, with timestamps preserved throughout for accurate mapping back to the video.

4.3 EVIDENCE GATHERING, REASONING AND VERDICT GENERATION:

Once a unique and verifiable claim has been extracted from the transcript, it enters the evidence gathering and verification stage. This is the most critical part of the pipeline as it determines whether the claim made in the video is supported by real-world information, contradicted by it, or whether the available evidence is insufficient to reach a conclusion. The stage consists of three sequential steps: parallel evidence retrieval from multiple trusted sources, evidence ranking and summarization, and final verdict generation using a large language model.

Evidence Gathering:

Evidence is gathered from five external sources simultaneously using a parallel execution approach, meaning all sources are queried at the same time rather than one after another. This design choice significantly reduces the overall response time for each claim. The five sources used are:

- **Wikipedia:** The claim is submitted as a search query to the Wikipedia API, which identifies the most relevant article and returns its title and summary text. Wikipedia provides a broad and reliable general knowledge base covering science, history, geography, politics, and a wide range of other topics, making it well suited as a primary evidence source for most factual claims.
- **SerpAPI:** This service queries Google and other search engines in real time and returns the results as structured data. Each result includes a title and a snippet of relevant text from the source page. SerpAPI provides broad web coverage and is particularly useful for claims involving recent events, current affairs, public figures, and topics that may not yet be documented on Wikipedia.
- **Tavily:** Tavily is an AI-focused search API that goes beyond standard keyword matching by retrieving and summarising web content in a semantically aware manner. It is designed to understand the intent behind a query and return the most contextually relevant content, making it a useful complement to the more keyword-driven SerpAPI results.
- **ArXiv:** ArXiv is an open-access repository of research papers covering fields such as physics, mathematics, computer science, biology, and economics. For claims that touch on scientific or technical topics, ArXiv provides access to peer-reviewed and

preprint academic literature, lending a higher degree of credibility to the evidence gathered.

- NewsAPI: NewsAPI aggregates articles from hundreds of news sources and blogs in real time. It is particularly valuable for verifying claims related to current events, political developments, and breaking news, where information may be too recent to appear on Wikipedia or in academic sources.

By combining these five sources, the system ensures that evidence is drawn from a diverse set of perspectives and information types, reducing the risk of missing relevant context that any single source alone might not cover.

Evidence Ranking and Summarization:

Once results are retrieved from all five sources, the individual text snippets are pooled together and ranked by their semantic similarity to the original claim using a sequence matching algorithm. This ranking step ensures that the most relevant pieces of evidence are prioritized and that the language model is not overwhelmed with loosely related information. The top three most relevant snippets are selected and concatenated into a single evidence block that is passed to the language model. If the total length of the retrieved evidence falls below a minimum threshold, meaning the sources returned very little useful content, the system automatically assigns an Inconclusive verdict without querying the language model at all. This prevents the model from generating unreliable verdicts based on insufficient information.

Verdict Generation:

The selected evidence block is passed to the Llama 3.3 70B Versatile model along with the original claim. The model is prompted to follow a structured reasoning process: it first identifies the factual assertion being made, then looks for supporting evidence, then looks for contradicting evidence, and finally arrives at a verdict based on the overall balance of what was found. Clear decision rules are provided in the prompt. A verdict of Support is assigned when there is strong evidence backing the claim with no meaningful contradiction. A verdict of Contradict is assigned when there is strong opposing evidence that clearly conflicts with the claim. A verdict of Inconclusive is assigned when the evidence is weak, mixed, or does not directly address the claim. The model is explicitly instructed to be conservative and default to Inconclusive whenever it is uncertain, which is an intentional design choice to

avoid presenting false confidence to the viewer. The model returns its response as a structured JSON object containing the verdict and a concise explanation of the reasoning behind it. This result is stored alongside the claim and its video timestamp and sent back to the Chrome extension to be displayed in the overlay.

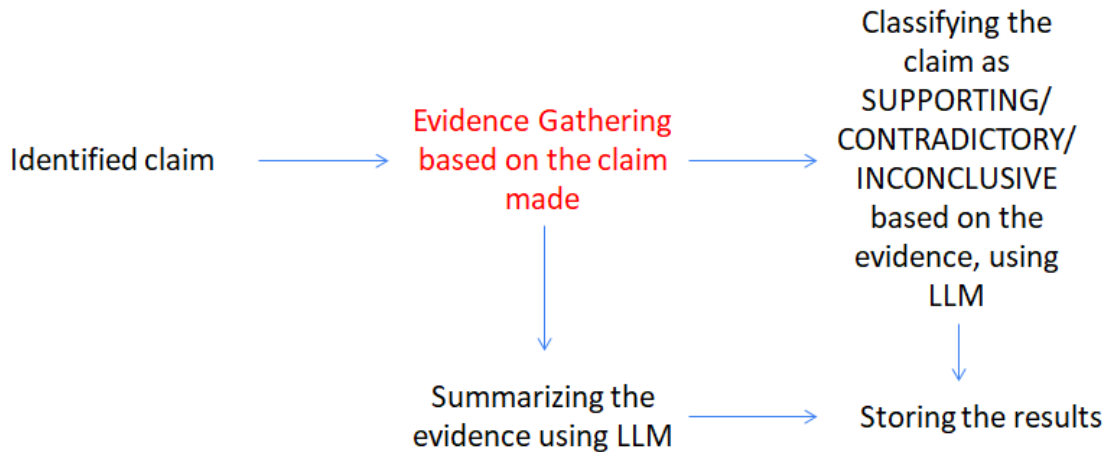


Fig 3: Evidence Gathering and Verdict Generation Pipeline

The figure illustrates the second half of the backend pipeline, showing how an identified claim moves through parallel evidence gathering from multiple sources, evidence summarization, LLM-based reasoning, and finally verdict classification as Supporting, Contradictory, or Inconclusive before the result is stored and returned to the extension.

4.4 EVALUATION ON THE FEVER DATASET:

To assess the accuracy of the verification pipeline independently of the speech processing components, the evidence gathering and verdict generation stages were evaluated against the FEVER dataset, a widely used benchmark for automated fact verification.

Dataset:

FEVER (Fact Extraction and Verification) is a large-scale publicly available dataset designed specifically for the task of claim verification. It was constructed by extracting sentences from Wikipedia and having human annotators mutate them into new claims, which were then verified against the original Wikipedia source and labelled accordingly. The dataset contains the following key characteristics:

- Over 185,000 annotated claims in total across training, development, and test splits
- Three label categories: Supports, Refutes, and Not Enough Information
- Claims span a wide range of topics including science, history, geography, politics, entertainment, and sports
- Each claim is paired with evidence sentences extracted from Wikipedia that justify the assigned label
- The labelled development split used for this evaluation contains 19,998 claim-label pairs
- Claims are deliberately diverse in difficulty, ranging from straightforward factual assertions to more nuanced statements requiring multi-hop reasoning

For this evaluation, a sample of 1000 claims was drawn from the labelled development split. The original FEVER labels were mapped to the three verdict categories used by FactLens, with Supports mapped to Support, Refutes mapped to Contradict, and Not Enough Information mapped to Inconclusive.

Preprocessing

Each claim was passed through the full evidence gathering pipeline using Wikipedia, ArXiv, SerpAPI, Tavily, NewsAPI. The retrieved evidence was ranked by similarity to the claim and the top three most relevant snippets were selected before being sent to the language model for verdict generation. A one second delay was applied between each request to stay within API rate limits during the evaluation run.

Evaluation Results:

The pipeline achieved an overall accuracy of 93% on the 1000 claim evaluation sample. The model performed strongly across all three verdict categories, correctly identifying the majority of supported and contradicted claims. Most of the errors involved claims being classified as Inconclusive rather than being misclassified between Support and Contradict, which is the preferred failure mode for a fact-checking system. Defaulting to Inconclusive when uncertain is significantly safer than confidently producing the wrong verdict, as it avoids misleading the viewer. The high accuracy on the FEVER benchmark validates the effectiveness of the combined evidence retrieval and reasoning pipeline used in FactLens.

4.5 REAL-TIME OVERLAY DISPLAY AND USER INTERFACE:

Once the backend completes the verification process for a batch of claims, the results are sent back to the Chrome extension and displayed to the viewer without any manual interaction required. The user interface is built entirely using HTML, CSS, and JavaScript and is designed to be non-intrusive, readable, and easy to understand at a glance.

Sending Results to the Frontend:

After the backend returns a response containing the list of verified claims, the background service worker receives the JSON data and immediately forwards it to the content script running on the active YouTube tab. Each claim object in the response contains the claim text, the absolute video timestamp at which it was spoken, the verdict, and a concise explanation of the reasoning behind the verdict. This data is passed to the overlay rendering logic without any additional processing, keeping the display as close to real time as possible.

Overlay Design and Structure:

The overlay is created as a fixed-position panel anchored to the top right corner of the YouTube page, sitting above all other page elements so it is always visible regardless of what the viewer is doing on the page. It has a dark semi-transparent background that keeps it visually distinct from the video without completely blocking the underlying content. The overlay includes a close button at the top that allows the viewer to dismiss it at any time if they find it distracting.

Each verified claim is rendered as a structured card inside the overlay containing the following information:

- **Claim:** The exact verifiable statement extracted from the video audio.
- **Timestamp:** The point in the video, shown in seconds, at which the claim was made, allowing the viewer to refer back to the exact moment if needed.
- **Verdict:** Displayed in color-coded text to allow for instant visual recognition. Green is used for Support, red for Contradict, and white for Inconclusive.
- **Summary:** A brief explanation of why the verdict was assigned, giving the viewer enough context to understand the reasoning without overwhelming them with detail.

New claims are added to the top of the overlay as they arrive, so the most recently verified claim is always the first one the viewer sees. Older claims remain visible below it, allowing

the viewer to scroll through the history of verified claims from the current session if they wish.

Key Design Decisions:

The overlay is intentionally kept lightweight and unobtrusive. It does not pause the video, does not require any clicks to update, and does not redirect the viewer away from the page. The structured layout with color coding ensures that a viewer can absorb the verdict at a glance without needing to read the full explanation. Claims are displayed live during video playback, meaning the overlay updates continuously as the video plays and new chunks are processed, giving the experience of near real-time fact checking alongside the viewing session.



Fig 4: FactLens Verification Overlay on YouTube

The figure shows the FactLens overlay displayed on a YouTube video page, presenting three verified claims extracted from a video about Putin. The first claim regarding an assassination attempt on Putin is labelled as Support in green, the second claim about his security being linked to assassination experiences is labelled as Inconclusive in white, and the third claim about Putin having six layers of security is labelled as Contradict in red, with each card

showing the timestamp at which the claim was spoken and a brief reasoning summary beneath the verdict.

4.6 DEPLOYMENT: BACKEND TUNNELING AND CHROME EXTENSION DISTRIBUTION:

Once the backend pipeline and the Chrome extension were fully developed and tested, the system was deployed in a straightforward manner that avoids the overhead of cloud infrastructure while still making the backend accessible over the internet during active use.

Backend Deployment using Ngrok:

The backend is run locally using Uvicorn, which serves the FastAPI application on port 8000. To make this locally running backend reachable from the Chrome extension over the internet, ngrok is used as a secure tunneling service. When ngrok is started, it establishes a secure HTTPS tunnel from a publicly accessible URL to the locally running backend server. This public URL is what the Chrome extension uses to send audio chunks and receive verified claim results.

The deployment process involves two steps running in parallel. First, the backend server is started using Uvicorn, which loads the Whisper model into memory and begins listening for incoming requests. Second, ngrok is launched pointing to the same port, which generates a public HTTPS URL of the form `https://xxxxxxx.ngrok-free.app`. This URL is then updated in the Chrome extension to point to the correct backend endpoint. The entire setup takes under a minute and requires no cloud account, no server configuration, and no domain management.

Ngrok also provides a local web interface that allows monitoring of all incoming requests in real time, which is useful for debugging and verifying that audio chunks are being received and processed correctly during testing.

Chrome Extension Distribution:

Since the extension is not published on the Chrome Web Store, it is packaged and distributed as a zipped folder. Publishing an extension on the Chrome Web Store requires a one-time developer registration fee of five US dollars, which was avoided given that the current scope of the project is limited to academic demonstration and evaluation rather than public release.

For this purpose, direct distribution as a zipped folder is a practical and cost-free alternative that works just as effectively.

The extension folder containing all the necessary files including the manifest, the background service worker, the content script, the popup interface, and all associated assets is compressed into a single ZIP archive. This archive can be shared directly with anyone who needs to use or evaluate the extension.

To install the extension from the ZIP file, the recipient extracts the folder and navigates to the Chrome extensions page by entering `chrome://extensions` in the browser address bar. Developer Mode is enabled using the toggle in the top right corner, and the extracted folder is loaded using the Load Unpacked option. The extension then appears in the Chrome toolbar and is ready to use immediately.

This distribution method is well suited for academic submissions, project demonstrations, and evaluator access, as it requires no publishing fees, no review process, and no internet connection to install. The only requirement for the extension to function is that the backend must be running and ngrok must be active at the time of use, with the correct public URL configured in the extension.

CHAPTER 5

RESULTS AND DISCUSSION

This chapter presents the outcomes of the FactLens system as a whole, covering the performance of the verification pipeline, the behaviour of the end-to-end system during real use, and a discussion of the strengths and limitations observed.

5.1 PIPELINE PERFORMANCE:

The core verification pipeline of FactLens, consisting of evidence gathering from multiple trusted sources and verdict generation using the Llama 3.3 70B Versatile model, was evaluated against the FEVER dataset to establish a quantitative measure of accuracy. The evaluation was conducted on a sample of 1000 claims drawn from the labelled development split of the dataset, covering a diverse range of topics including science, history, politics, geography, and entertainment.

The pipeline achieved an overall accuracy of 93% across all three verdict categories of Support, Contradict, and Inconclusive. The model correctly classified 311 out of 331 supported claims, 285 out of 312 contradicted claims, and 334 out of 357 inconclusive claims. The majority of errors involved claims being assigned an Inconclusive verdict rather than the correct Support or Contradict label, which is a deliberate and acceptable failure mode for a fact-checking system. Assigning Inconclusive when uncertain is significantly safer than confidently producing a wrong verdict, as it avoids actively misleading the viewer. These results validate that the evidence retrieval and reasoning components of FactLens are robust and reliable when operating on well-formed factual claims.

5.2 END-TO-END SYSTEM BEHAVIOUR:

When tested on real YouTube videos, the system demonstrated the ability to extract meaningful and verifiable claims from spoken audio and return verdict results within a reasonable time window after each audio chunk was processed. The overlay updated continuously during playback, displaying new claims as they were verified without requiring any interaction from the user.

The claim detection stage successfully filtered out a large proportion of non-factual content including opinions, jokes, and casual remarks, ensuring that only statements worth verifying were passed through to the evidence gathering stage. The context window maintained across chunks allowed the model to correctly resolve pronouns and references in many cases, producing cleaner and more accurate claim statements compared to processing each segment in isolation.

The duplicate filtering mechanism proved effective in preventing the same claim from appearing multiple times in the overlay, which would have been a significant usability issue given that speakers often repeat or rephrase the same point across multiple sentences. Claims that were sufficiently similar to a previously extracted one were silently discarded, keeping the overlay concise and relevant.

The colour coded verdict display in the overlay allowed viewers to instantly identify the credibility status of each claim without needing to read the full explanation. The green, red, and white colour scheme for Support, Contradict, and Inconclusive respectively provided a clear and intuitive visual signal that required minimal cognitive effort from the user.

5.3 OBSERVED STRENGTHS:

The following strengths were observed during testing and evaluation:

- The multi-source evidence approach combining Wikipedia, SerpAPI, Tavily, ArXiv, and NewsAPI provided broad coverage across general knowledge, current events, and academic topics, reducing the likelihood of missing relevant evidence for any given claim.
- Parallel evidence retrieval kept response times manageable despite querying five sources simultaneously.
- The conservative verdict strategy of defaulting to Inconclusive when evidence is insufficient or mixed helped maintain the trustworthiness of the system by avoiding overconfident wrong verdicts.
- The non-intrusive overlay design allowed viewers to consume fact-checking results without pausing the video or leaving the YouTube page.
- The FEVER evaluation score of 93% demonstrates that the verification pipeline performs at a level comparable to dedicated fact-checking systems, despite being embedded within a broader real-time pipeline.

5.4 LIMITATIONS AND CHALLENGES:

Despite the strong results, several limitations were identified during the development and testing of FactLens:

- The system currently supports English language content only, as the Whisper model is configured for English transcription. Videos in other languages are not processed correctly.
- There is an inherent delay between when a claim is made in the video and when the verified result appears in the overlay, as the system waits for a full 10 second audio chunk before processing begins. This means the system is near real-time rather than truly real-time.
- The accuracy of claim detection depends heavily on the quality of the audio. Videos with background music, heavy accents, multiple speakers, or poor recording quality may produce inaccurate transcriptions, which in turn affects the quality of the extracted claims.
- The ngrok-based deployment means the backend must be actively running on a machine for the extension to function. If the machine is shut down or ngrok is closed, the extension stops working entirely.
- The session store that maintains context and tracks previously seen claims is held in memory and is lost if the backend restarts, which could cause duplicate claims to reappear in a new session covering the same video.
- API rate limits across the evidence gathering services can occasionally cause one or more sources to return no results for a given claim, which may push the verdict toward Inconclusive even when sufficient information exists elsewhere.

5.5 DISCUSSION:

The results demonstrate that FactLens successfully addresses the core gaps identified in existing systems. Unlike prior approaches that classify content at the video level, FactLens operates at the individual claim level, providing specific and actionable information to the viewer. Unlike text-only fact-checking systems, it handles spoken audio directly through transcription. And unlike offline detection models, it delivers results during active video playback without requiring the user to leave the platform or take any additional steps.

The 93% accuracy on FEVER, combined with the observed behaviour during real video testing, suggests that the pipeline is capable of producing reliable verdicts for a wide range of factual claims. The system is not intended to be the final word on whether a claim is true or false, but rather to serve as a passive assistant that draws the viewer's attention to statements that may warrant further investigation, backed by evidence from trusted sources. This positions FactLens as a practical and accessible tool for improving the critical media literacy of everyday YouTube viewers.

CHAPTER 6

CONCLUSION

This project presented FactLens, a Chrome extension designed to automatically detect and verify factual claims made in YouTube videos in near real time. The system was built to address a clear and growing problem: the ease with which misinformation spreads through online video content and the lack of accessible tools that help everyday viewers evaluate the accuracy of what they are watching.

The end-to-end pipeline covers every stage of the fact-checking process, from capturing audio directly from the browser, transcribing it using the Whisper base model, extracting verifiable claims using the Llama 3.3 70B Versatile model, gathering evidence from five trusted external sources in parallel, and delivering a reasoned verdict through a non-intrusive overlay on the YouTube page. The entire process runs without requiring any manual effort from the user beyond enabling the extension.

The verification pipeline was evaluated against the FEVER dataset and achieved an accuracy of 93% across 1000 claims, demonstrating that the evidence retrieval and reasoning components perform reliably across a diverse range of topics. The conservative approach of defaulting to Inconclusive when evidence is insufficient ensures the system does not mislead viewers with overconfident wrong verdicts.

Several areas remain open for future improvement, including support for non-English content, reducing processing delay for closer to true real-time performance, replacing the in-memory session store with persistent storage, and publishing the extension on the Chrome Web Store once the registration cost is feasible.

Overall, FactLens demonstrates that automated evidence-grounded fact-checking can be embedded directly into the video watching experience in a way that is practical and accessible, representing a meaningful step toward helping viewers engage more critically with the content they consume every day.

REFERENCES

1. Google, "Chrome Extensions Manifest V3 Overview," Google Developers Documentation, 2023. [Online]. Available: <https://developer.chrome.com/docs/extensions/mv3/intro/>
2. A. Radford, J. W. Kim, T. Xu, G. Brockman, C. McLeavey, and I. Sutskever, "Robust Speech Recognition via Large-Scale Weak Supervision," OpenAI Technical Report, 2022. [Online]. Available: <https://openai.com/research/whisper>
3. Meta AI, "Llama 3.3 70B Versatile Model Card," Meta AI Research, 2024. [Online]. Available: <https://ai.meta.com/blog/meta-llama-3/>
4. SerpApi, "SerpApi: Google Search API Documentation," SerpApi, 2024. [Online]. Available: <https://serpapi.com/>
5. Tavily, "Tavily AI Search API Documentation," Tavily, 2024. [Online]. Available: <https://tavily.com/>
6. S. Langa, "Uvicorn: An ASGI Web Server Implementation for Python," Uvicorn Documentation, 2023. [Online]. Available: <https://www.uvicorn.org/>
7. Ngrok Inc., "Ngrok: Secure Tunnels to Localhost," Ngrok Documentation, 2024. [Online]. Available: <https://ngrok.com/docs/>
8. R. Cao, "Online Misinformation Detection in Live Streaming Videos," in Proc. IEEE International Conference on Multimedia and Expo (ICME), 2025.
9. X. Yang, Y. Liu, Z. Wang, and H. Chen, "Fake News Detection in Short Videos by Integrating Semantic Credibility and Multi-Granularity Contrastive Learning," IEEE Transactions on Multimedia, 2025.
10. J. Kim, S. Park, and M. Lee, "A Hybrid Detection Method for YouTube Fake News Using Related Video Data," Expert Systems with Applications, vol. 238, 2025.
11. T. Thorne, A. Vlachos, C. Christodoulopoulos, and A. Mittal, "FEVER: A Large-Scale Dataset for Fact Extraction and VERification," in Proc. North American Chapter of the Association for Computational Linguistics (NAACL), 2018, pp. 809–819.
12. W. Y. Wang, "LIAR: A Benchmark Dataset for Fake News Detection," in Proc. Annual Meeting of the Association for Computational Linguistics (ACL), 2017, pp. 422–426.

13. N. Hassan, C. Li, and M. Tremayne, "Detecting Check-Worthy Factual Claims in Presidential Debates," in Proc. ACM International Conference on Information and Knowledge Management (CIKM), 2015, pp. 1835–1838.
14. Y. Qi, J. Xu, and Z. Zhang, "FakeSV-VLM: A Vision-Language Model for Fake Short Video Detection," in Proc. ACM International Conference on Multimedia (MM), 2024.
15. H. Liu, X. Wang, and T. Chen, "VMID: Video Misinformation Detection via Large Language Model Fusion," in Proc. IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 2024.
16. Z. Chen, Y. Li, and J. Zhang, "A Consistency-Aware Model for Multimodal Misinformation Detection," in Proc. International Joint Conference on Artificial Intelligence (IJCAI), 2024.
17. Groq Inc., "Groq API Documentation: Fast AI Inference," Groq, 2024. [Online]. Available: <https://console.groq.com/docs/>

10% Overall Similarity

The combined total of all matches, including overlapping sources, for each database.

Match Groups

- 65 Not Cited or Quoted 9%
Matches with neither in-text citation nor quotation marks
- 6 Missing Quotations 1%
Matches that are still very similar to source material
- 5 Missing Citation 1%
Matches that have quotation marks, but no in-text citation
- 0 Cited and Quoted 0%
Matches with in-text citation present, but no quotation marks

Top Sources

- 5% Internet sources
- 4% Publications
- 9% Submitted works (Student Papers)

Integrity Flags

0 Integrity Flags for Review

Our system's algorithms look deeply at a document for any inconsistencies that would set it apart from a normal submission. If we notice something strange, we flag it for you to review.

A Flag is not necessarily an indicator of a problem. However, we'd recommend you focus your attention there for further review.

Juvraj Singh

28/04/26